

Assignment 3: Ajax, JSON, Frontend Frameworks, CSS Frameworks, RWD, Node.js, and Ticketmaster API

Rubric (aka Grading Guidelines)

Graders will test the homework using the latest version of Google Chrome

Total Points: 15

- **Navbar (0.25 points in total)**
 - The navbar should have the same element and behave similar to the reference video - **0.25 points**.
- **Routing (0.5 points in total)**
 - Three routes in the address bar should be implemented - **0.25 points** per route.
(If a route in the address bar does not change or reloading a page shows a different page – 0 points)
- **Events Search Form (4 points in total)**
 - Autocomplete field should display suggestions – **0.25 points**
 - Autocomplete field should display entered value as the first option – **0.25 points**
 - Autocomplete field should have a “clear icon” that clears the selection – **0.25 points**
 - Autocomplete field should have a loading indication icon – **0.25 points**
 - The layout of the form should be proper and should contain all the input fields as shown in the reference video – **0.5 point**
 - Proper error messages should be displayed on clicking submit button for invalid/empty inputs – **1 points**
 - On checking the auto-detect location checkbox, location field should get clear and disable - **0.5 points**
 - Auto-detect functionality should return a correct location – **0.5 points**
 - Manually entered location should be geocoded correctly – **0.5 points**

- Events results grid (1.25 points in total)

Display the events information retrieved from the API service in a tabular format as shown in the reference video or homework description

- The result records should be present in a grid view of cards with all the information present and displayed correctly - **0.25 points**
- Data populated in the table should be correctly sorted in ascending order of date/time – **0.25 points**
- The result card should have a favorite button that reflects current state and behaves as expected – **0.25 points**
- On clicking anywhere on a card, it should direct to the details page with the correct event - **0.25 points**.
- If no results are returned - a correct message is shown– **0.25 points**

- Events details page (4.5 points in total)

- Event header (1 point in total)
 - The event name should be present – **0.1 points**
 - The page should have 3 tabs that behave as expected – **0.25 points**
 - The “Buy tickets” should be present and should open a correct link in a new tab – **0.25 points**
 - The favorite button should be present and should reflect current state and work as described – **0.25 points**
 - The back arrow should be present and should work as expected – **0.15 points**
- Event details tab (1 point in total)
 - All fields and a seat map should be present as shown in the reference video or homework description - **0.25 points**
 - Event date and time should be correct regardless of a client timezone and match one on the TicketMaster “Buy tickets” page – **0.25 points**
 - Ticket status field should be displayed in proper color - **0.2 points**
 - On sale: Green
 - Off sale: Red
 - Canceled: Black
 - Postponed: Orange
 - Rescheduled: Orange
 - Twitter icon should be present, it should tweet as shown in the reference video or in the snapshot in homework description - **0.15 points**
 - Facebook icon should be present, it should create a post as shown in the reference video or in the snapshot in homework description - **0.15 points**

- Artists/Teams tab (1.25 points in total)
 - Display a cover image and an artist name– **0.1 points**.
 - Display “followers” and “popularity” with the correct formatting – **0.25 points**
 - Display a “Open in Spotify” button that opens the artist’s profile on Spotify in a new tab - **0.15 points**
 - Display a grid of artist’s albums – **0.25 points**
 - Each album should be a link that opens the corresponding album on Spotify in a new tab – **0.25 points**
 - For events that are not related to music, the Artist/Teams tab should be disabled – **0.25 points**
- Venue Tab (1.25 points in total)
 - Display all necessary attributes related to event’s venue as shown in the snapshot in homework description or the reference video – **0.5 points**.
 - Address field in Venue should be a concatenation of address, city, state, separated by comma - **0.25 points**
 - The address line should be a link that opens a Google map with the location marker pointing to the event’s location in a new tab – **0.25 points**
 - The “See events” button should link to TicketMaster website with the list of events of the correct venue – **0.25 points**
- List of Favorites (1.5 points in total)
 - Favorite event’s information must be displayed in a grid format - **0.25 points**
 - Favorite events should be persisted and updated both from database (on page reload) and dynamically (on event “like” action) – **0.5 points**
 - On clicking Trash icon, ‘Removed from favorites’ alert should get displayed and that event should get deleted from the grid of favorite events – **0.5 point**
 - If no favorite events are available, display ‘**No favorite events to show**’ message as shown in reference video - **0.25 points**
- Responsiveness (2 points in total)
 - In the mobile browser, the navbar and all routes should be the same as the screenshots provided in the description and the reference video - **1 point**.
 - In the mobile browser, all functionalities must work exactly - **1 point**.
- AI Log (1 point)

See the “AI Log Rubric” for details. Submit file process_log.txt.

Scoring will be split as follows:

 - Prompt (Clarity) (0.25 points)

- Issues (Specificity) (0.25 points)
- Fix (Correctness) (0.25 points)
- Explanation (Depth) (0.25 points)

****IMPORTANT****

- You may call the Ipinfo API and Google Geocoding API from a browser, but you should not call the Ticketmaster API directly from your frontend. Doing this will result in a **4 points** penalty.
- List of favorite events should be retrieved from the backend. It should return added events even if the browser is closed and reopened. **2 points** penalty otherwise.
- The window should not reload or load html parts for any kind of data request. We focus on building single-page applications for this assignment. Refreshing the page will result in a **3 points** penalty.
- You should implement HW3 using a GCP cloud service as described in Assignment 2. Otherwise, **4 points** will be deducted. This will be verified with an additional link in the Table of exercises, showing a sample API call to your “cloud” service and verifying the corresponding JSON result.
- **Source code submission**
Submit your source code files to DEN D2L as a single ZIP archive. Include separate subfolders for your frontend and backend source code, plus any additional files needed to build your app (e.g., yaml file). The timestamp of your last submission will be used to verify if you used any grace days. Make sure you don't upload the `node\_modules` in the zip file and also make sure `package.json` is included in the zip file.